AML LAB ASSIGNMENT – 1

Q1.

a. Implement linear regression model, apply regularization methods (Ridge and Lasso).

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

data = load_diabetes()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=1.0)
}
```

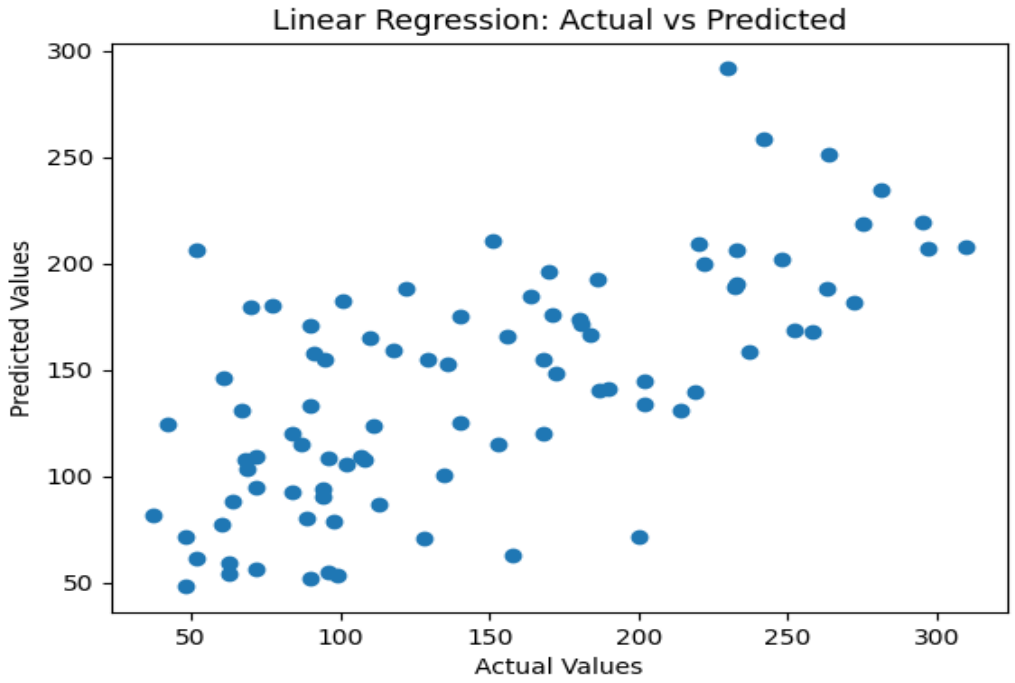
```
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print("\\n" + name)
    print("MSE:", mean_squared_error(y_test, y_pred))
    print("R2 Score:", r2_score(y_test, y_pred))
    print("Coefficients:", model.coef_)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

plt.scatter(y_test, y_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()
```

output:

```
\nLinear Regression
MSE: 2900.1936284934827
R2 Score: 0.45260276297191926
Coefficients: [  1.75375799 -11.51180908  25.60712144  16.82887167 -44.44885564
  24.64095356   7.67697768  13.1387839   35.16119521   2.35136365]
\nRidge Regression
MSE: 2892.0145657501726
R2 Score: 0.45414652070698225
Coefficients: [  1.80734179 -11.44818951  25.73269892  16.73429974 -34.67195409
  17.05307485   3.36991411  11.76426044  31.3783838    2.45813922]
\nLasso Regression
MSE: 2824.568094049959
R2 Score: 0.46687670944102466
Coefficients: [ 0.68703212 -9.29751904 26.21922482 15.65731357 -8.2281718  -0.
 -9.02408714  3.42086059 22.63646533  2.09864712]
```



b. Implementation of classification using decision trees. Application of splitting criteria such as entropy, information gain, and Gini impurity.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Breast Cancer dataset

data = load_breast_cancer()


X = data.data

y = data.target

print("Dataset Shape:", X.shape)

print("Target Classes:", data.target_names)
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# Entropy calculation
def entropy(y):
    classes, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)

    return -np.sum(probabilities * np.log2(probabilities))

# Gini impurity calculation
def gini(y):
    classes, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)

    return 1 - np.sum(probabilities ** 2)

# Information Gain calculation
def information_gain(parent, left, right):
    parent_entropy = entropy(parent)

    weighted_entropy = (
```

```
        len(left) / len(parent) * entropy(left)
        + len(right) / len(parent) * entropy(right)
    )

    return parent_entropy - weighted_entropy

# Example calculation
parent = np.array([0, 0, 1, 1, 1, 0])
left = np.array([0, 0, 0])
right = np.array([1, 1, 1])

print("\nEntropy:", entropy(parent))
print("Gini Impurity:", gini(parent))
print("Information Gain:", information_gain(parent, left, right))

# Decision Tree using Gini
model_gini = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

model_gini.fit(X_train, y_train)

y_pred_gini = model_gini.predict(X_test)

print("\nDecision Tree - Gini Impurity")
print("Accuracy:", accuracy_score(y_test, y_pred_gini))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_gini))
```

```
print("Classification Report:\n",
      classification_report(y_test, y_pred_gini,
                            target_names=data.target_names))

# Decision Tree using Entropy
model_entropy = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

model_entropy.fit(X_train, y_train)

y_pred_entropy = model_entropy.predict(X_test)

print("\nDecision Tree - Entropy")
print("Accuracy:", accuracy_score(y_test, y_pred_entropy))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_entropy))
print("Classification Report:\n",
      classification_report(y_test, y_pred_entropy,
                            target_names=data.target_names))

# Visualize Decision Tree
plt.figure(figsize=(20, 10))

plot_tree(
    model_gini,
    feature_names=data.feature_names,
    class_names=data.target_names,
```

```
    filled=True,
    rounded=True,
    max_depth=3
)
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
```

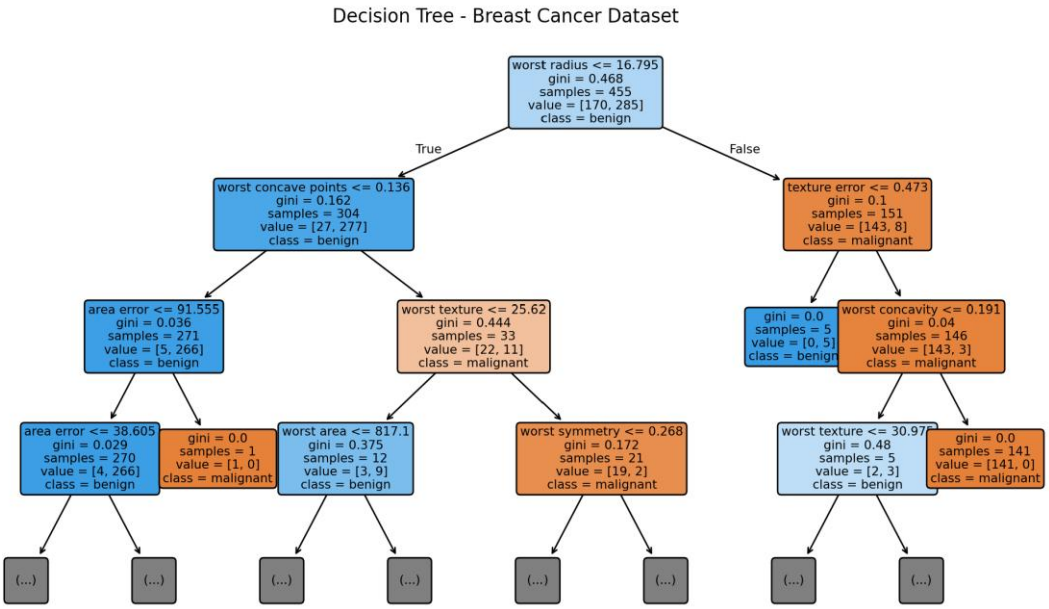
output:

```
    accuracy                           0.91       114
   macro avg       0.90      0.92      0.91       114
weighted avg       0.92      0.91      0.91       114


Decision Tree - Entropy
Accuracy: 0.9122807017543859
Confusion Matrix:
 [[40  2]
 [ 8 64]]
Classification Report:
              precision    recall  f1-score   support

   malignant       0.83      0.95      0.89        42
      benign       0.97      0.89      0.93        72

    accuracy                           0.91       114
   macro avg       0.90      0.92      0.91       114
weighted avg       0.92      0.91      0.91       114
```



c. Implement logistic regression model, apply sigmoid function, decision boundary. Apply regularization methods (Ridge and Lasso).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report

# 1. Load dataset
data = load_breast_cancer()

X = data.data
y = data.target
print("Dataset Shape:", X.shape)
print("Target Classes:", data.target_names)

# 2. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("\nTraining Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)

# 3. Feature scaling
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

print("\nSigmoid Function")
print("Sigmoid(0):", sigmoid(0))
print("Sigmoid(2):", sigmoid(2))
print("Sigmoid(-2):", sigmoid(-2))

# 5. Logistic Regression without regularization penalty
# Scikit-learn uses L2 by default, so penalty=None is specified.
logistic = LogisticRegression(
    penalty=None,
    max_iter=5000
)
logistic.fit(X_train, y_train)
y_pred = logistic.predict(X_test)
print("\nLogistic Regression")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n",
      classification_report(y_test, y_pred,
                            target_names=data.target_names))
```

```
# 6. Ridge Regularization (L2)
ridge = LogisticRegression(
    penalty="l2",
    C=1.0,
    max_iter=5000
)

ridge.fit(X_train, y_train)
ridge_pred = ridge.predict(X_test)

print("\nRidge Regularization (L2)")
print("Accuracy:", accuracy_score(y_test, ridge_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, ridge_pred))

# 7. Lasso Regularization (L1)
lasso = LogisticRegression(
    penalty="l1",
    solver="liblinear",
    C=1.0,
    max_iter=5000
)
lasso.fit(X_train, y_train)

lasso_pred = lasso.predict(X_test)
print("\nLasso Regularization (L1)")
print("Accuracy:", accuracy_score(y_test, lasso_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, lasso_pred))
```

```
# 8. Display coefficients
print("\nModel Coefficients")
print("Logistic Regression:", logistic.coef_)
print("Ridge Coefficients:", ridge.coef_)
print("Lasso Coefficients:", lasso.coef_)

# 9. Calculate predicted probabilities
probabilities = ridge.predict_proba(X_test)
print("\nPredicted Probabilities (First 5):")
print(probabilities[:5])

# 10. Plot Sigmoid Function
z = np.linspace(-10, 10, 100)

plt.figure(figsize=(7, 5))
plt.plot(z, sigmoid(z))
plt.axhline(y=0.5, linestyle="--")
plt.axvline(x=0, linestyle="--")
plt.xlabel("Input (z)")
plt.ylabel("Probability")
plt.title("Sigmoid Function")
plt.grid()
plt.show()

# 11. Decision Boundary using two features
# Use mean radius and mean texture
```

```
X_2 = data.data[:, [0, 1]]


X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_2, y, test_size=0.2, random_state=42, stratify=y
)
scaler2 = StandardScaler()
X_train2 = scaler2.fit_transform(X_train2)
X_test2 = scaler2.transform(X_test2)


boundary_model = LogisticRegression(
    penalty="l2",
    C=1.0,
    max_iter=5000
)
boundary_model.fit(X_train2, y_train2)


# Create mesh grid
x_min, x_max = X_train2[:, 0].min() - 1, X_train2[:, 0].max() + 1
y_min, y_max = X_train2[:, 1].min() - 1, X_train2[:, 1].max() + 1


xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)
# Predict over grid
Z = boundary_model.predict(
    np.c_[xx.ravel(), yy.ravel()]
)
```

```
Z = Z.reshape(xx.shape)


# Plot decision boundary
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
plt.scatter(
    X_train2[:, 0],
    X_train2[:, 1],
    c=y_train2,
    cmap="coolwarm",
    edgecolors="black"
)
plt.xlabel("Mean Radius (Scaled)")
plt.ylabel("Mean Texture (Scaled)")
plt.title("Logistic Regression Decision Boundary")
plt.show()
```

output:

```
Dataset Shape: (569, 30)
Target Classes: ['malignant' 'benign']

Training Shape: (455, 30)
Testing Shape: (114, 30)

Sigmoid Function
Sigmoid(0): 0.5
Sigmoid(2): 0.8807970779778823
Sigmoid(-2): 0.11920292202211755

Logistic Regression
Accuracy: 0.9210526315789473
Confusion Matrix:
 [[40  2]
 [ 7 65]]
Classification Report:
              precision    recall  f1-score   support

   malignant       0.85      0.95      0.90        42
      benign       0.97      0.90      0.94        72

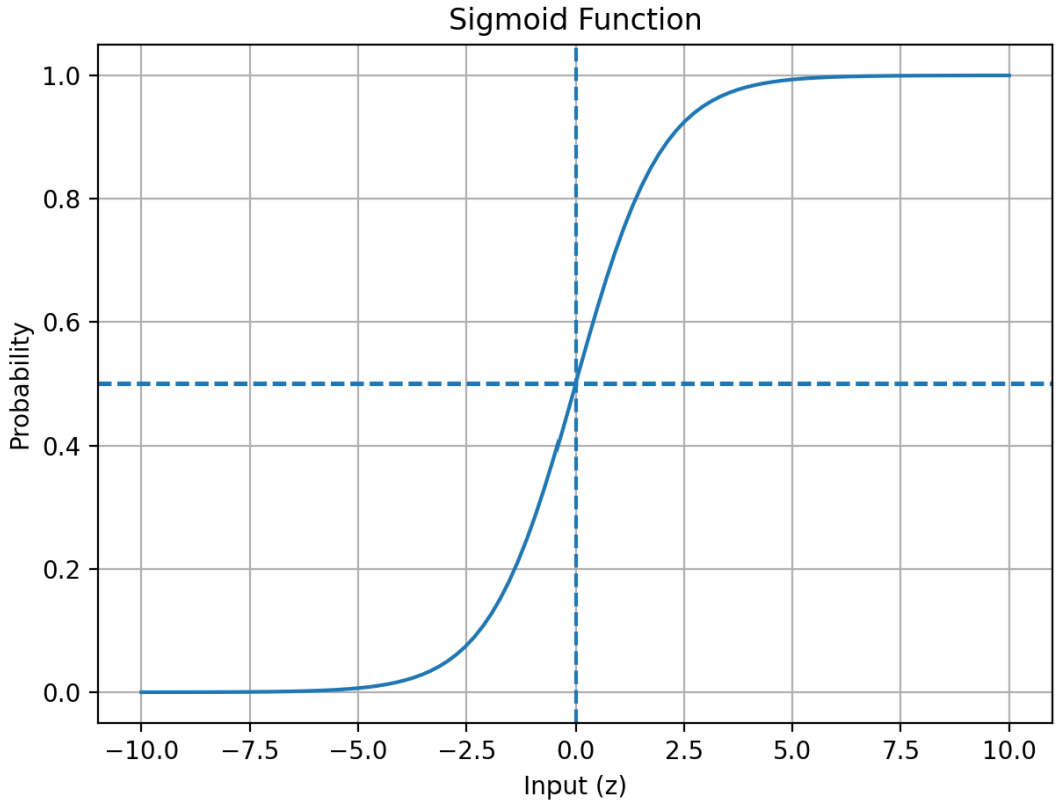
    accuracy                           0.92       114
   macro avg       0.91      0.93      0.92       114
weighted avg       0.93      0.92      0.92       114

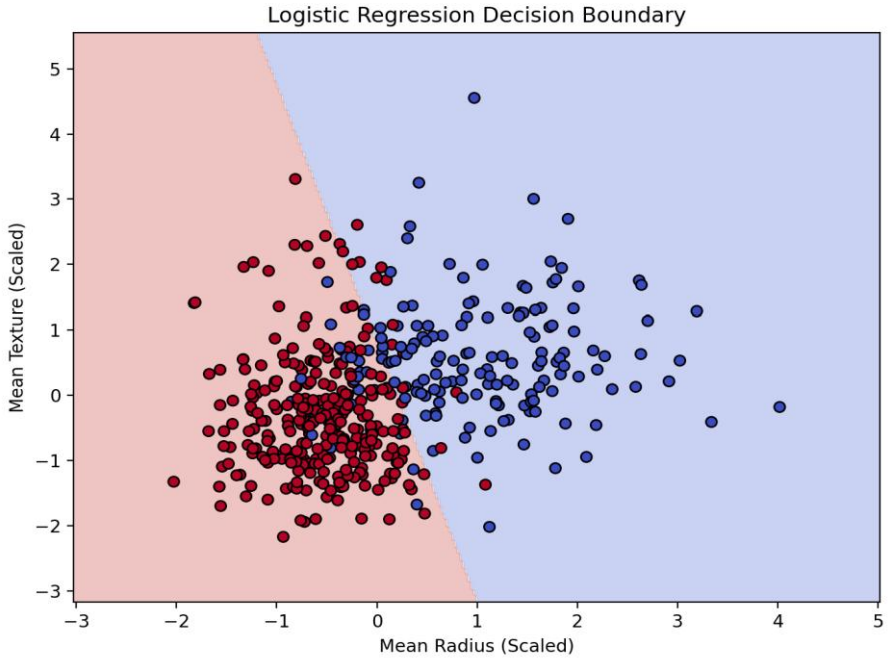

Ridge Regularization (L2)
Accuracy: 0.9824561403508771
Confusion Matrix:
 [[41  1]
 [ 1 71]]
```

```
Lasso Regularization (L1)
Accuracy: 0.9912280701754386
Confusion Matrix:
 [[41  1]
 [ 0 72]]

Model Coefficients
Logistic Regression: [[  17.5772907   -98.50930373   15.64775932  -35.59520108  -53.59564097
   360.41216752  -98.48866555  -35.33746617  -19.31688792 -107.35464015
  -168.37160641   36.04249855  129.6982943  -204.43529105   39.42045665
  -108.6944328   233.11851066 -165.57574554   55.91019566   44.71828014
   -59.43579157  -25.793332     8.50838649 -165.8654877   -31.02078081
    24.92697088 -262.56920494  -22.16936167  -61.51311486   39.77303826]]
Ridge Coefficients: [[-0.51147901 -0.55269775 -0.47629789 -0.54105924 -0.21247927  0.64834159
  -0.60210291 -0.70415649 -0.16723273  0.19973173 -1.08296534  0.24882301
  -0.54433323 -0.92910402 -0.16027571  0.64722656  0.16056252 -0.44378424
   0.36049156  0.43789426 -0.94761615 -1.25508804 -0.76322007 -0.9477559
  -0.74662481  0.05551412 -0.82315065 -0.95368636 -0.93918141 -0.1872508 ]]
Lasso Coefficients: [[ 0.         -0.39746687  0.          0.          0.          0.
   0.         -0.49098299  0.          0.08166047 -2.16798636  0.02473575
   0.          0.         -0.19841668  0.69775783  0.         -0.07227858
   0.          0.1910208  -1.24505065 -1.20495575  0.         -3.53366296
  -0.60941194  0.         -0.67712789 -1.70650913 -0.66397735  0.        ]]

Predicted Probabilities (First 5):
[[9.99999941e-01 5.88824186e-08]
 [1.13351672e-05 9.99988665e-01]
 [9.93589175e-01 6.41082462e-03]
 [4.66491463e-01 5.33508537e-01]
 [9.99999999e-01 6.52500097e-10]]
```





d. Apply K-NNN algorithms for the practical problems.

```
# Import libraries

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix, classification_report

from sklearn.metrics import ConfusionMatrixDisplay


# 1. Load dataset

data =
pd.read_csv(r"C:\Users\omgow\OneDrive\Desktop\MSIS\AML_LAB\Datasets\Social_Network_Ads.csv")

print("First 5 rows:")

print(data.head())
```

```
print("\nDataset Shape:", data.shape)
# 2. Select input features and target
X = data[['Gender', 'Age', 'EstimatedSalary']].copy()
y = data['Purchased']

# 3. Convert Gender into numerical values
X['Gender'] = X['Gender'].map({
    'Female': 0,
    'Male': 1
})

# 4. Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# 5. Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 6. Create KNN model
```

```
knn = KNeighborsClassifier(n_neighbors=5)

# 7. Train the model
knn.fit(X_train, y_train)

# 8. Predict test data
y_pred = knn.predict(X_test)

# 9. Evaluate the model
print("\nKNN Classification Results")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(    y_test, y_pred,target_names=['Not Purchased', 'Purchased']))

# 10. Test different K values
k_values = range(1, 21)
accuracies = []
for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    acc = accuracy_score(y_test, prediction)
    accuracies.append(acc)

    print("K =", k, "Accuracy =", acc)
```

```
# 11. Find best K
best_k = list(k_values)[accuracies.index(max(accuracies))]

print("\nBest K:", best_k)
print("Best Accuracy:", max(accuracies))

# 12. Plot K vs Accuracy
plt.figure(figsize=(8, 5))
plt.plot(k_values, accuracies, marker='o')
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("KNN - K Value vs Accuracy")
plt.xticks(list(k_values))
plt.grid()
plt.show()
# 13. Confusion Matrix Visualization
ConfusionMatrixDisplay.from_predictions( y_test,  y_pred,display_labels=['Not Purchased', Purchased'], cmap='Blues')
plt.title("KNN Confusion Matrix")
plt.show()

# 14. Predict a new customer
new_customer = pd.DataFrame({
    'Gender': [1],
    'Age': [35],
    'EstimatedSalary': [60000]
})
new_customer_scaled = scaler.transform(new_customer)
```

```
prediction = knn.predict(new_customer_scaled)

print("\nNew Customer Prediction:")

if prediction[0] == 1:

    print("Customer will purchase the product")

else:

    print("Customer will not purchase the product")
```

Output:

```
Training Data: (320, 3)
Testing Data: (80, 3)

KNN Classification Results
Accuracy: 0.9

Confusion Matrix:
[[46  5]
 [ 3 26]]

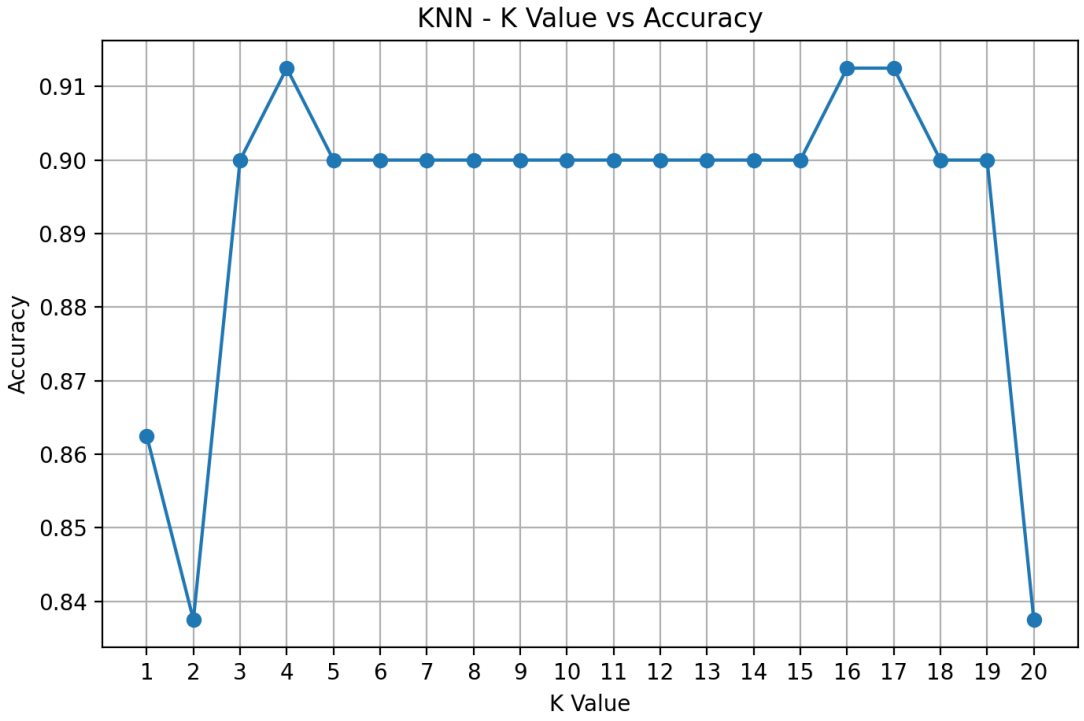
Classification Report:
              precision    recall  f1-score   support

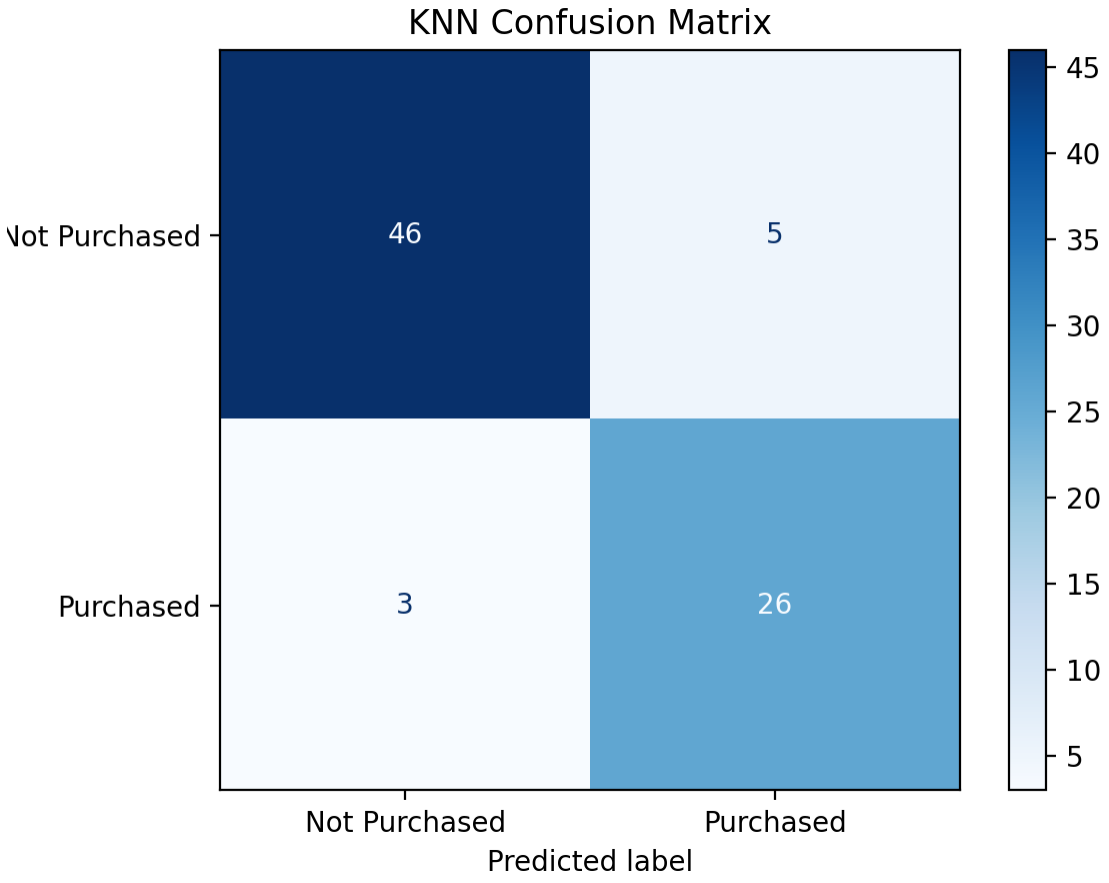
Not Purchased       0.94      0.90      0.92        51
    Purchased       0.84      0.90      0.87        29

     accuracy                           0.90        80
    macro avg       0.89      0.90      0.89        80
 weighted avg       0.90      0.90      0.90        80
```

```
K = 1 Accuracy = 0.8625
K = 2 Accuracy = 0.8375
K = 3 Accuracy = 0.9
K = 4 Accuracy = 0.9125
K = 5 Accuracy = 0.9
K = 6 Accuracy = 0.9
K = 7 Accuracy = 0.9
K = 8 Accuracy = 0.9
K = 9 Accuracy = 0.9
K = 10 Accuracy = 0.9
K = 11 Accuracy = 0.9
K = 12 Accuracy = 0.9
K = 13 Accuracy = 0.9
K = 14 Accuracy = 0.9
K = 15 Accuracy = 0.9
K = 16 Accuracy = 0.9125
K = 17 Accuracy = 0.9125
K = 18 Accuracy = 0.9
K = 19 Accuracy = 0.9
K = 20 Accuracy = 0.8375

Best K: 4
Best Accuracy: 0.9125
```


KNN - K Value vs Accuracy
Accuracy
0.91
0.90
0.89
0.88
0.87
0.86
0.85
0.84
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
K Value


KNN Confusion Matrix
Not Purchased
Purchased
46
5
3
26
Not Purchased
Purchased
Predicted label
45
40
35
30
25
20
15
10
5

e. Implement Naive Bayes model for classification.

```
# Import libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix, classification_report

# 1. Load dataset
data =
pd.read_csv(r"C:\Users\omgow\OneDrive\Desktop\MSIS\AML_LAB\Datasets\diabetes.csv")
print("First 5 rows:")
print(data.head())
print("\nDataset Shape:", data.shape)

# 2. Separate features and target
X = data.drop("Outcome", axis=1)
y = data["Outcome"]
print("\nFeatures:")
print(X.columns)
print("\nTarget Distribution:")
print(y.value_counts())

# 3. Handle invalid zero values
# Zero is not a valid measurement for these attributes
```

```
zero_columns = [ "Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]

X[zero_columns] = X[zero_columns].replace(0, np.nan)


print("\nMissing Values:")

print(X.isnull().sum())


# 4. Split dataset

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y)


print("\nTraining Shape:", X_train.shape)

print("Testing Shape:", X_test.shape)


# 5. Create Naive Bayes model

# Median imputation handles missing values

model = make_pipeline( SimpleImputer(strategy="median"), GaussianNB())


# 6. Train model

model.fit(X_train, y_train)

# 7. Predict test data

y_pred = model.predict(X_test)


# 8. Evaluate model

print("\nNaive Bayes Classification Results")

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")

print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred, target_names=["No Diabetes", "Diabetes"]))

# 9. Predict probability
y_probability = model.predict_proba(X_test)

print("\nPredicted Probabilities (First 5):")
print(y_probability[:5])

# 10. Predict a new patient
new_patient = pd.DataFrame([{
    "Pregnancies": 2,
    "Glucose": 120,
    "BloodPressure": 70,
    "SkinThickness": 25,
    "Insulin": 100,
    "BMI": 28.5,
    "DiabetesPedigreeFunction": 0.5,
    "Age": 30
}])

prediction = model.predict(new_patient)
print("\nNew Patient Prediction:")
if prediction[0] == 1:
    print("Predicted Class: Diabetes")
else:
    print("Predicted Class: No Diabetes")
print("Prediction Probabilities:", model.predict_proba(new_patient))
```

output:

```
First 5 rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6      148             72             35        0  33.6                     0.627   50        1
1            1       85             66             29        0  26.6                     0.351   31        0
2            8      183             64              0        0  23.3                     0.672   32        1
3            1       89             66             23       94  28.1                     0.167   21        0
4            0      137             40             35      168  43.1                     2.288   33        1

Dataset Shape: (768, 9)

Features:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age'],
      dtype='object')

Target Distribution:
Outcome
0    500
1    268
Name: count, dtype: int64
```

```
Missing Values:
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
dtype: int64

Training Shape: (614, 8)
Testing Shape: (154, 8)

Naive Bayes Classification Results
Accuracy: 0.7012987012987013

Confusion Matrix:
[[74 26]
 [20 34]]

Classification Report:
              precision    recall  f1-score   support

 No Diabetes       0.79      0.74      0.76       100
    Diabetes       0.57      0.63      0.60        54

    accuracy                           0.70       154
   macro avg       0.68      0.68      0.68       154
weighted avg       0.71      0.70      0.70       154


Predicted Probabilities (First 5):
[[0.48890357 0.51109643]
 [0.63713118 0.36286882]
 [0.81278109 0.18721891]
 [0.89937121 0.10062879]
 [0.98636366 0.01363634]]

New Patient Prediction:
Predicted Class: No Diabetes
Prediction Probabilities: [[0.96145938 0.03854062]]
```